



fit@hcmus

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

SOCKET PROGRAMMING

Sinh Viên Thực Hiện:

Phạm Tấn Phát – 20120636
Nhan Hữu Hiếu – 21120070
Trần Huỳnh Gia Bảo – 24120267

Giáo Viên Hướng Dẫn:

ThS. Huỳnh Thụy Bảo Trân
ThS. Chung Thùy Linh

Thành phố Hồ Chí Minh, tháng 12 năm 2025

Mục lục

THÔNG TIN NHÓM	3
1 TRIỂN KHAI GIAO THỨC RTSP TRÊN CLIENT VÀ RTP TRÊN SERVER	4
1.1 Tổng quan về hệ thống	4
1.2 Phân tích gói tin RTP (RtpPacket.py)	4
1.2.1 Cấu trúc Header	5
1.2.2 Phương thức hoạt động	5
1.2.3 Thiết kế RTSP Client (Client.py)	6
1.2.4 Xử lý dữ liệu và Giao diện	7
1.3 Kết luận	8
2 MỞ RỘNG: HD STREAMING	9
2.1 Tổng quan yêu cầu	9
2.2 Phân tích vấn đề	9
2.2.1 Giới hạn MTU và nhu cầu fragmentation	9
2.2.2 Thách thức Kỹ thuật	10
2.3 Hướng xử lý	10
2.3.1 Phía Server: Phân mảnh (Fragmentation)	10
2.3.2 Phía Client: Tái lắp ráp (Reassembly)	11
2.3.3 Network Quality Monitoring	12
2.3.4 Mở rộng hỗ trợ các định dạng MJPEG khác	12
2.3.5 Một số vấn đề khác và hướng giải quyết	12
2.4 Kết luận	13
3 MỞ RỘNG: CLIENT-SIDE CACHING	14
3.1 Giới thiệu	14
3.2 Kiến trúc tổng quan	15
3.2.1 JitterBuffer	15
3.2.2 FrameAssembler	15
3.2.3 LRU Frame Cache	15
3.3 Luồng xử lý dữ liệu	16
3.4 Pseudocode	16
3.4.1 Pseudocode JitterBuffer	16

3.4.2	Pseudocode FrameAssembler	16
3.4.3	Pseudocode ClientCache	17
3.5	Kết luận	17

THÔNG TIN NHÓM

Đồ án được thực hiện bởi Nhóm 13.2, lớp Mạng máy tính - 24CTT3. Bao gồm các thành viên:

1. Phạm Tấn Phát. MSSV: 20120636.
2. Nhan Hữu Hiếu. MSSV: 21120070.
3. Trần Huỳnh Gia Bảo. MSSV: 24120267.

Chương 1

TRIỂN KHAI GIAO THỨC RTSP TRÊN CLIENT VÀ RTP TRÊN SERVER

1.1 Tổng quan về hệ thống

Hệ thống được thiết kế theo mô hình Client-Server để truyền tải video (dưới dạng các frame ảnh MJPEG). Hệ thống sử dụng hai giao thức mạng chính:

- **RTSP (Real-Time Streaming Protocol):** Dùng để điều khiển phiên truyền thông (như cái điều khiển từ xa: Setup, Play, Pause, Teardown). Giao thức này chạy trên nền TCP để đảm bảo tin cậy.
- **RTP (Real-Time Transport Protocol):** Dùng để đóng gói và vận chuyển dữ liệu video thực tế. Giao thức này chạy trên nền UDP để tối ưu hóa tốc độ và chấp nhận mất gói tin nhỏ trong thời gian thực.

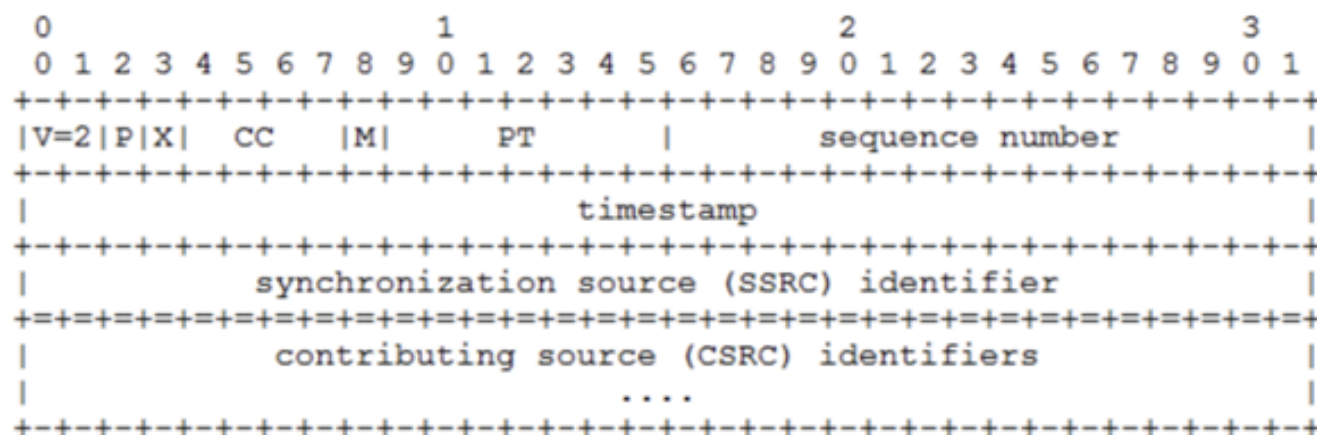
1.2 Phân tích gói tin RTP (RtpPacket.py)

Phân tích gói tin RTP (RtpPacket.py)

1.2.1 Cấu trúc Header

Theo mã nguồn, kích thước Header cố định là 12 bytes. Cấu trúc bao gồm các trường quan trọng sau:

- **Version (V):** 2 bits. Phiên bản RTP (thường là 2).
- **Sequence Number:** 16 bits. Số thứ tự của gói tin, giúp Client sắp xếp lại các gói tin bị đảo lộn hoặc phát hiện mất gói. Được tính bằng cách dịch bit: $seqnum = self.header[2] \ll 8 | self.header[3]$.
- **Timestamp:** 32 bits. Thời gian lấy mẫu, giúp đồng bộ hóa khi phát lại.
- **Payload Type (PT):** 7 bits. Định dạng dữ liệu (trong code Server quy định là 26 - MJPEG).



1.2.2 Phương thức hoạt động

- **Encode:** Server đóng gói dữ liệu bằng cách gán các giá trị vào mảng byte (bytearray). Ví dụ: header[0] chứa Version, Padding, Extension và CSRC Count.
- **Decode:** Client tách 12-byte đầu tiên làm header, phần còn lại là payload (dữ liệu ảnh).

1.2.3 Thiết kế RTSP Client (Client.py)

Client đóng vai trò là trình phát video, quản lý giao diện người dùng (GUI) và xử lý luồng mạng.

1.2.3.1 Máy trạng thái (Finite State Machine)

Client hoạt động dựa trên các trạng thái để quản lý logic điều khiển:

- **INIT**: Trạng thái khởi tạo, chưa kết nối stream.
- **READY**: Đã thiết lập kết nối (SETUP thành công), sẵn sàng phát hoặc đã dừng (PAUSE).
- **PLAYING**: Đang nhận và hiển thị dữ liệu video.

1.2.3.2 . Quản lý kết nối

Client duy trì hai loại kết nối socket riêng biệt:

- **RTSP Socket (TCP)**: Dùng để gửi các lệnh điều khiển (SETUP, PLAY, PAUSE, TEARDOWN). Kết nối này được thiết lập ngay khi khởi tạo Client.
- **RTP Socket (UDP)**: Dùng để nhận dữ liệu video. Socket này được tạo khi Client nhận phản hồi SETUP thành công từ Server và bắt đầu lắng nghe trên cổng đã định.

1.2.3.2 . Quy trình hoạt động

A. SETUP:

- Client gửi bản tin SETUP chứa tên file video và cổng RTP của Client (ví dụ: Transport: RTP/UDP; client_port=...).
- Server khởi tạo phiên làm việc (Session ID) và luồng đọc video (VideoStream), sau đó trả về trạng thái 200 OK.
- Chuyển trạng thái: INIT -> READY.

B. PLAY:

- Client gửi lệnh PLAY.
- Server bắt đầu luồng gửi dữ liệu (Thread sendRtp).
- Client tạo một luồng riêng (*threading.Thread(target = self.listenRtp)*) để liên tục nhận các gói tin UDP, giải mã RTP packet để lấy payload (ảnh) và cập nhật lên GUI.
- Chuyển trạng thái: READY -> PLAYING.

C. PAUSE:

- Client gửi lệnh PAUSE.
- Server nhận lệnh và set cờ sự kiện (*event.set()*) để dừng vòng lặp gửi RTP.
- Client ngừng cập nhật frame hình ảnh.
- Chuyển trạng thái: PLAYING -> READY.

D. TEARDOWN:

- Client gửi lệnh TEARDOWN để kết thúc phiên.
- Xóa các file cache tạm thời (*os.remove*) và đóng socket.
- Chuyển trạng thái: Về INIT hoặc thoát chương trình.

1.2.4 Xử lý dữ liệu và Giao diện

- **Bộ đệm (Buffering/Cache):** Khi nhận được payload từ RTP, Client ghi dữ liệu ra một file ảnh tạm thời (*cache- < sessionID > .jpg*).
- **Hiển thị:** Thư viện PIL (Pillow) được dùng để mở file ảnh này và hiển thị lên Tkinter Label.
- **Đồng bộ:** Client kiểm tra seqNum của gói tin đến. Nếu *currFrameNbr > self.frameNbr*, gói tin được chấp nhận; ngược lại, nếu gói tin đến muộn (số thứ tự nhỏ hơn), nó sẽ bị loại bỏ để tránh hiện tượng giật lùi hình ảnh.

1.3 Kết luận

Hệ thống đã hiện thực thành công mô hình truyền tải video cơ bản:

1. Tách biệt rõ ràng giữa kênh điều khiển (RTSP/TCP) và kênh dữ liệu (RTP/UDP).
2. Xử lý đóng gói và giải mã gói tin RTP đúng chuẩn cơ bản (Header 12 bytes).
3. Client xử lý đa luồng (Multithreading) để đảm bảo giao diện không bị treo khi nhận dữ liệu mạng.

Chương 2

MỞ RỘNG: HD STREAMING

2.1 Tổng quan yêu cầu

Phần mở rộng HD Video Streaming nhằm nâng cao khả năng của hệ thống streaming video, cho phép truyền tải video độ phân giải cao (720p, 1080p) thông qua giao thức RTP/UDP.

Mục tiêu chính:

- Triển khai cơ chế phân mảnh (fragmentation) frame HD thành các RTP packets phù hợp với giới hạn MTU.
- Đảm bảo playback mượt mà với độ trễ thấp.
- Giám sát và phân tích số lượng mất frame và chất lượng mạng trong quá trình streaming.

2.2 Phân tích vấn đề

2.2.1 Giới hạn MTU và nhu cầu fragmentation

Mạng Ethernet chuẩn có MTU khoảng 1500 bytes, trong khi đó một frame MJPEG ở độ phân giải HD thường có kích thước:

- 720p: 50-100 KB
- 1080p: 100-200 KB

Sự chênh lệch này yêu cầu mỗi frame phải được chia nhỏ thành nhiều RTP packets. Để đáp ứng việc phân mảnh và tái lắp ráp (reassembly), ta cần có một cơ chế đồng bộ chính xác để đảm bảo tính toàn vẹn và đầy đủ của dữ liệu được truyền đi.

2.2.2 Thách thức Kỹ thuật

Về phía Server:

- Phân mảnh frame sao cho mỗi packet không vượt quá giới hạn an toàn (1400 bytes).
- Đảm bảo các fragments của cùng một frame có chung timestamp.
- Đánh dấu rõ ràng fragment cuối cùng của mỗi frame.

Về phía Client:

- Tập hợp đúng các fragments thuộc cùng một frame.
- Xử lý trường hợp packet loss mà không làm crash hệ thống.
- Phân biệt giữa khoảng trống do PAUSE/PLAY và packet loss thực sự.

2.3 Hướng xử lý

2.3.1 Phía Server: Phân mảnh (Fragmentation)

Ta cần áp dụng phương pháp chia frame thành các segments có kích thước phù hợp, ở đồ án này, nhóm chọn kích thước cố định là **1400 bytes**. Quy trình như sau:

1. Đọc frame hoàn chỉnh từ video file.
2. Sử dụng hàm *fragmentPayload()* để chia frame thành danh sách các fragments.

-
3. Gán timestamp chung cho tất cả fragments của frame đó.
 4. Gửi tuần tự các fragments, đánh dấu marker *bit* = 1 cho fragment cuối.
 5. Tăng sequence number sau mỗi fragment được gửi.

Lý do chọn 1400 bytes: MTU Ethernet thường là 1500 bytes, trong đó:

- IP header chiếm khoảng 20 bytes;
- UDP header chiếm khoảng 8 bytes;
- RTP header chiếm khoảng 12 bytes;

Còn lại khoảng 1460 bytes. Để an toàn, nhóm chọn payload tối đa là **1400 bytes**.

2.3.2 Phía Client: Tái lắp ráp (Reassembly)

Client sử dụng buffer động để tập hợp fragments dựa trên timestamp:

1. Nhận RTP packet, giải mã để lấy timestamp và marker bit.
2. Nếu timestamp khác với frame hiện tại, bắt đầu frame mới.
3. Nếu timestamp trùng, thêm payload vào buffer hiện tại.
4. Khi gặp marker *bit* = 1 tập hợp toàn bộ fragments thành frame hoàn chỉnh.
5. Hiện thị frame và reset buffer.

Cơ chế đồng bộ:

- Sử dụng `fragmentLock` để đảm bảo thread-safe.
- Flag `shouldClearBuffer` để xóa buffer khi PAUSE/PLAY, tránh hiện thị frame cũ.

2.3.3 Network Quality Monitoring

Hệ thống giám sát liên tục các chỉ số:

- Packet loss rate: Số packet bị mất / Tổng packet mong đợi.
- Bitrate: $(\text{Tổng bytes} \times 8) / \text{Thời gian truyền}$.
- Network status: Phân loại 4 mức GOOD, FAIR, UNSTABLE, POOR.

Hệ thống sẽ có cảnh báo phù hợp cho người dùng khi network status không ổn định, dựa vào tỉ lệ mất gói và tốc độ mạng theo thời gian thực.

2.3.4 Mở rộng hỗ trợ các định dạng MJPEG khác

Theo đề bài, ngoài định dạng yêu cầu trong lab (movie.Mjpeg), chương trình còn cần hỗ trợ thêm dạng MJPEG khác.

VideoStream.py được mở rộng để hỗ trợ 2 định dạng MJPEG:

Proprietary format (movie.Mjpeg):

- Mỗi frame có 5-byte header chứa frame length.
- Format: [5-byte length][JPEG data][5-byte length][JPEG data]...

Standard MJPEG:

- Nhận diện frame qua JPEG markers: 0xFFD8 (start) và 0xFFD9 (end).
- Phù hợp với file MJPEG chuẩn từ các nguồn online.

2.3.5 Một số vấn đề khác và hướng giải quyết

- Các fragments đến không đồng thời, có thể bị trộn lẫn giữa các frames.

Giải pháp: Sử dụng timestamp làm khóa của group. Tất cả fragments có cùng timestamp thuộc một frame. Chỉ khi gặp marker *bit* = 1 mới reassemble.

- Khi *PAUSE* rồi *PLAY*, fragments cũ còn trong buffer có thể được ghép với fragments mới, tạo ảnh lỗi, giật lag, không xem được.

Giải pháp: Implement flag *shouldClearBuffer* với thread lock. Mỗi khi state chuyển, buffer được reset hoàn toàn.

- Khi người dùng *PAUSE* lâu rồi *PLAY*, sequence number gap rất lớn, dễ nhầm lẫn thành packet loss, dẫn đến báo lỗi về network status sai.

Giải pháp: Đặt threshold 100 packets. Gap lớn hơn được coi là session mới, không tính vào loss rate.

- Khi video kết thúc và network timeout thì cả hai trường hợp, client đều không nhận packets, cần phân biệt để hiển thị message phù hợp.

Giải pháp: Track consecutiveTimeouts và kiểm tra *frameNbr* > 0. Nếu đã nhận frames trước đó và timeout kéo dài 3 giây, coi là video ended.

2.4 Kết luận

- Phần HD Video Streaming đã được triển khai thành công với đầy đủ chức năng fragmentation, reassembly và network monitoring. Hệ thống có khả năng streaming ổn định video 720p và 1080p trên mạng LAN với packet loss rate thấp (< 0.3% cho tất cả các lần test).
- Cơ chế timestamp-based grouping kết hợp marker bit đảm bảo tính đầy đủ của frames. Thread-safe buffer management giải quyết tốt vấn đề đồng bộ khi state transitions. Real-time statistics cung cấp khả năng giám sát và debug hiệu quả.
- Giải pháp đáp ứng đầy đủ yêu cầu đề bài: fragmentation cho frames vượt MTU, playback mượt mà với độ trễ thấp, và phân tích chi tiết frame loss cũng như network usage.

Chương 3

MỞ RỘNG: CLIENT-SIDE CACHING

3.1 Giới thiệu

Trong streaming video qua mạng, hiện tượng jitter là một vấn đề phổ biến, ảnh hưởng trực tiếp đến trải nghiệm người dùng.

Mặc dù RTP/UDP được thiết kế cho real-time streaming với độ trễ thấp, nhưng nếu tốc độ mạng không ổn định sẽ có thể khiến các packets đến không đều, gây ra hiện tượng giật lag hoặc freeze trong quá trình phát video.

Client-side caching là một kỹ thuật quan trọng để giải quyết vấn đề này bằng cách tạo một buffer trung gian giữa quá trình nhận dữ liệu từ mạng và hiển thị video. Thay vì hiển thị frame ngay khi nhận được, client sẽ pre-buffer một số lượng frames nhất định trước khi bắt đầu phát, giúp giảm tối đa các biến động về thời gian truyền tải.

Mục tiêu chính:

Phần client-side caching trong đề án này tập trung vào hai chức năng chính:

Pre-buffering khi SETUP:

- Sau khi nhận lệnh SETUP thành công, client không phát ngay mà tích lũy N frames trong buffer
- Khi người dùng nhấn PLAY, video bắt đầu phát từ buffer đã chuẩn bị sẵn
- Giảm thiểu startup delay và đảm bảo khởi động mượt mà

Playback từ buffer:

- Trong quá trình PLAY, client duy trì buffer với N frames ahead
- Hiển thị video từ buffer thay vì trực tiếp từ RTP stream
- Buffer được nạp liên tục để bù đắp cho các frame đã phát

3.2 Kiến trúc tổng quan

Client-side caching gồm 3 thành phần chính:

3.2.1 JitterBuffer

- Lưu packet theo seqnum.
- Sắp xếp lại thứ tự packet.
- Giữ packet tối đa 150 ms để đợi packet trễ.
- Trả ra danh sách packet đã sẵn sàng để assemble.

3.2.2 FrameAssembler

- Ghép các fragment có cùng timestamp.
- Khi timestamp thay đổi hoặc marker $bit = 1 \rightarrow$ tạo frame hoàn chỉnh.
- Giúp xây dựng frame HD từ nhiều packet nhỏ.

3.2.3 LRU Frame Cache

- Lưu frame hoàn chỉnh theo cơ chế LRU để tránh đầy RAM.
- Đưa key frame vào *frame_queue* để GUI lấy.
- Tách biệt hoàn toàn luồng xử lý dữ liệu và luồng hiển thị.

3.3 Luồng xử lý dữ liệu

1. Server gửi frame HD → chia thành nhiều packet RTP.
2. Client nhận packet → gọi *on_rtp_packet()*.
3. JitterBuffer reorder packet và xử lý packet trễ.
4. FrameAssembler ghép frame hoàn chỉnh.
5. Cache lưu frame và đưa vào queue.
6. GUI hiển thị frame từ queue.

3.4 Pseudocode

3.4.1 Pseudocode JitterBuffer

```
function on_packet(seq, payload, info):  
    buffer[seq] = packet  
    ready_packets = []  
    while next sequential seq exists:  
        pop → ready_packets  
    if oldest packet > max_hold_ms:  
        pop → ready_packets  
    return ready_packets
```

3.4.2 Pseudocode FrameAssembler

```
function add_packet(seq, payload, timestamp):  
    if timestamp != current_timestamp:  
        frame = concatenate(parts)  
        reset parts[]  
        start new frame
```

```
parts.append(payload)
if marker == 1:
    return full_frame
```

3.4.3 Pseudocode ClientCache

```
function on_rtp_packet(seq, payload, info):
    ready = jitter.pop_ready()
    for packet in ready:
        frame = assembler.add_packet(packet)
        if frame_complete:
            cache.put(frame)
            frame_queue.push(frame_key)
function get_frame():
    return next frame from frame_queue
```

3.5 Kết luận

Client-side caching đảm bảo:

- Ghép frame HD từ nhiều fragment RTP
- Chống jitter và reorder packet
- Tăng mượt playback (FPS ổn định)
- Giảm lag khi mạng yếu
- Hỗ trợ Pause/Play mượt
- Nâng cao trải nghiệm người xem